

# SQL注入漏洞分析与修复报告

Flask 用户管理系统

实验名称：SQL注入漏洞分析与修复

数据库类型：SQLite 3

实验日期：2026 年 7 月 8 日

作 者：葛敬茹

# 1 项目概述

## 1.1 项目介绍

本项目是一个基于 Python Flask 框架的用户信息管理系统，提供用户登录、注册、信息查询和用户搜索功能。系统使用 SQLite 作为数据库，通过 sqlite3 模块进行数据库操作。

## 1.2 数据库表结构

```
CREATE TABLE users (  
    id          INTEGER PRIMARY KEY AUTOINCREMENT,  
    username    TEXT UNIQUE NOT NULL,  
    password    TEXT NOT NULL,  
    email       TEXT,  
    phone       TEXT  
);
```

## 1.3 涉及数据库操作的接口

| 路由               | 方法   | 功能            | SQL 操作        |
|------------------|------|---------------|---------------|
| GET / (index 函数) | GET  | 首页展示、搜索用户     | SELECT        |
| GET /search      | GET  | 接收搜索关键词并跳转到首页 | SELECT (通过首页) |
| POST /register   | POST | 注册新用户         | INSERT        |
| 启动时 init_db()    | —    | 初始化数据库和默认用户   | INSERT        |

## 1.4 存在 SQL 注入的位置

经过对项目源代码的逐行审计，本次实验共发现 3 处 SQL 注入漏洞：

| 编号      | 文件     | 函数         | 路由             | SQL 类型 | 用户输入来源         |
|---------|--------|------------|----------------|--------|----------------|
| SQL-001 | app.py | init_db()  | 启动时            | INSERT | 硬编码数据（低风险）     |
| SQL-002 | app.py | index()    | GET /          | SELECT | URL 参数 keyword |
| SQL-003 | app.py | register() | POST /register | INSERT | 表单字段           |

## 2 SQL 注入漏洞概述

---

### 2.1 什么是 SQL 注入

SQL 注入 (SQL Injection) 是指攻击者通过在输入数据中插入恶意的 SQL 代码,使得应用程序在执行 SQL 查询时执行了非预期的 SQL 命令。当应用程序将用户输入直接拼接到 SQL 语句中,且未对输入进行任何过滤或转义时,就会产生 SQL 注入漏洞。

### 2.2 本项目产生 SQL 注入的原因

项目代码在 3 处使用了 Python 的 f-string 字符串格式化机制将变量直接嵌入 SQL 语句。例如:

```
sql = f"SELECT id, username, email, phone FROM users \
      WHERE username LIKE '{search_keyword}%' \
      OR email LIKE '{search_keyword}%'"
```

f-string 会将变量 `search_keyword` 的值直接替换到字符串中,对单引号、SQL 关键字等特殊字符不做任何转义处理。当 `search_keyword` 的值为 `' OR '1'='1'` 时,拼接后的 SQL 为:

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'
```

原本的 `LIKE '%keyword%'` 匹配逻辑被破坏, `OR '1'='1'` 使得 WHERE 子句永远为真。

### 2.3 本项目 SQL 注入的特点

- 注入类型:字符型注入。SQL 语句中使用单引号包裹字符串值,攻击者需要通过单引号闭合前引号。
- 数据库类型:SQLite 3。支持 `?` 占位符的参数化查询。
- **SQL 操作类型**:包含 SELECT (搜索功能)和 INSERT (注册功能)。
- 闭合方式:单引号 `'` 闭合。搜索功能 SQL 使用 `LIKE '%{keyword}%'`,注册功能 SQL 使用 `VALUES ('{username}', ...)`。

## 3 漏洞 SQL-002：搜索功能 SELECT 注入

### 3.1 漏洞位置

|    |                                    |
|----|------------------------------------|
| 文件 | app.py                             |
| 函数 | index()                            |
| 路由 | GET / ( 参数 keyword 来自 /search 跳转 ) |

对应代码（修复前）：

```
@app.route("/")
def index():
    username = session.get("username")
    ...
    search_keyword = request.args.get("keyword", "")
    if search_keyword:
        conn = sqlite3.connect("data/users.db")
        c = conn.cursor()
        sql = f"SELECT id, username, email, phone FROM users \
            WHERE username LIKE '%{search_keyword}%' \
            OR email LIKE '%{search_keyword}%'"
        c.execute(sql)
        search_results = c.fetchall()
        conn.close()
    return render_template("index.html", user=user, ...)
```

### 3.2 漏洞原理

搜索功能的执行路径为：用户访问 `/search?keyword=xxx`，`/search` 路由使用 `redirect(f"/?keyword={keyword}")` 将 `keyword` 参数传递给首页。首页的 `index()` 函数从 URL 参数中获取 `keyword`，然后通过 f-string 将其直接拼接到 `SELECT` 语句的 `LIKE` 子句中：

```
sql = f"SELECT id, username, email, phone FROM users \
    WHERE username LIKE '%{search_keyword}%' \
    OR email LIKE '%{search_keyword}%'"
```

f-string 的求值机制是直接将变量的值嵌入字符串，不进行任何转义。当 `search_keyword` 的值中包含单引号 `'` 时，该单引号会闭合 `LIKE` 子句开始的 `'%` 中的前引号，使得后续的 SQL 关键字可以被攻击者控制。

具体来说，keyword 的值 ' OR '1'='1' 被拼入后：

- `username LIKE '%'` —— 单引号闭合了 LIKE 模式，`%` 匹配任意字符串
- `OR '1'='1'` —— `'1'='1'` 是恒真条件，后面的 `%'` 被当作字符串比较的一部分
- `OR email LIKE '%'` `OR '1'='1'` —— 同样的结构作用于 email 字段
- 由于 WHERE 子句使用 OR 连接多个条件，只要任意一个条件为真，整行数据就会被返回

### 3.3 攻击方式

#### Payload 1 : OR 恒真条件

```
keyword = ' OR '1'='1'
URL 编码后: %27%20OR%20%271%27%3D%271
完整 URL: /search?keyword=%27%20OR%20%271%27%3D%271
```

作用：使 WHERE 子句永远为真，返回 users 表全部数据。

#### Payload 2 : UNION SELECT 自定义数据

```
keyword = ' UNION SELECT 1, 'inj', 'inj@x.com', '138' --
URL 编码后: %27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%27138%27--
完整 URL: /search?keyword=%27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%27138%27--
```

作用：UNION 合并第二个查询的结果，向搜索结果中插入攻击者自定义的数据。

注：第二个查询必须返回 4 列，与原始 SELECT 的 4 列 (id, username, email, phone) 一致。

#### Payload 3 : ORDER BY 探测列数

```
keyword = ' ORDER BY 5 --
URL 编码后: %27%20ORDER%20BY%205--
完整 URL: /search?keyword=%27%20ORDER%20BY%205--
```

作用：判断 SELECT 查询的列数，若 ORDER BY 5 成功则至少有 5 列。

ORDER BY 6 失败报错则确认最大列数为 5。

### 3.4 SQL 执行分析

#### 正常搜索 (keyword = admin)

攻击前 SQL (代码中拼接的结果)：

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%admin%' OR email LIKE '%admin%'
```

数据库执行方式：

SQLite 将 LIKE '%admin%' 解释为模糊匹配模式

→ 在 users 表中搜索用户名或邮箱包含 "admin" 的记录

查询结果：

返回一条记录：id=1, username=admin, email=admin@example.com, phone=13800138000

执行结论：查询结果正确，符合业务预期。

## OR 注入 ( keyword = ' OR '1'='1 )

攻击前 SQL ( 开发人员期望的 SQL )：

```
SELECT ... WHERE username LIKE '' OR '1'='1%' OR email LIKE '' OR '1'='1%'
```

攻击后 SQL ( f-string 实际拼接的结果 )：

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'
```

数据库执行方式：

SQLite 解析 WHERE 子句时按 OR 连接符拆分为 4 个条件：

- 条件1: username LIKE '%' → TRUE ( 所有非空字符串都匹配 '%' )
- 条件2: '1'='1%' → TRUE ( 字符串比较 '1'='1' 成立 )
- 条件3: email LIKE '%' → TRUE ( 所有非空字符串都匹配 '%' )
- 条件4: '1'='1%' → TRUE ( 同上 )

由于 SQLite 使用短路求值，条件1 为真即返回当前行。

查询结果：

返回 users 表中所有记录 ( admin、alice 及所有已注册用户 )

为什么执行成功：

1. 攻击者输入的单引号 ' 与 LIKE 模式开头的 '%' 中的前引号配对，闭合了 LIKE 子句
2. 后续的 OR '1'='1 成为独立的 SQL 条件表达式
3. 数据库在解析时认为这是 4 个 OR 连接的条件，而非原本的 2 个 LIKE 匹配
4. 恒真条件使所有行都被选中返回

## UNION SELECT 注入 ( keyword = ' UNION SELECT 1,'inj','inj@x.com','138'-- )

攻击后 SQL ( f-string 实际拼接的结果 )：

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' UNION SELECT 1,'inj','inj@x.com','138'--%'
                        ^^^^^^^^^^
```

注释符，后续 SQL 被忽略

数据库执行方式：

SQLite 将 SQL 解析为两个 SELECT 查询的 UNION：

- 查询1 ( 原始 )：SELECT id, username, email, phone FROM users WHERE username LIKE '%'
- 查询2 ( 注入 )：SELECT 1, 'inj', 'inj@x.com', '138'

UNION 将两个查询的结果集纵向合并

查询结果：

结果集包含原始用户数据 + 攻击者插入的 ( 1, 'inj', 'inj@x.com', '138' )

为什么执行成功：

1. 单引号 ' 闭合了 LIKE '%', 使 WHERE 子句提前结束
2. UNION SELECT 是合法的 SQL 语法, 用于合并两个查询结果
3. -- 注释符使后续的 '%' OR... 被忽略, SQL 语法完整
4. 第二个 SELECT 返回 4 列 (1, 'inj', 'inj@x.com', '138'), 与第一个 SELECT 的列数匹配
5. 数据库成功执行了 UNION 操作, 自定义数据出现在搜索结果中

### 3.5 漏洞影响

- 数据泄露: 攻击者通过 OR 注入可获取 users 表中全部注册用户的信息, 如用户名、邮箱、手机号。
- 任意数据查询: 通过 UNION SELECT 可查询数据库中任意表的数据, 如 `sqlite_master` 获取数据库结构。
- 信息收集: ORDER BY 探测可确定查询列数, 为构造 UNION SELECT 做准备。

## 4 漏洞 SQL-003：注册功能 INSERT 注入

### 4.1 漏洞位置

|    |                |
|----|----------------|
| 文件 | app.py         |
| 函数 | register()     |
| 路由 | POST /register |

对应代码（修复前）：

```
@app.route("/register", methods=["GET", "POST"])
def register():
    error = None
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")
        email = request.form.get("email", "").strip()
        phone = request.form.get("phone", "").strip()
        conn = sqlite3.connect("data/users.db")
        c = conn.cursor()
        sql = f"INSERT INTO users (username, password, email, phone) \
            VALUES ('{username}', '{password}', '{email}', '{phone}')"
        c.execute(sql)
        conn.commit()
        conn.close()
        return redirect("/login?msg=注册成功，请登录")
    return render_template("register.html", error=error)
```

### 4.2 漏洞原理

注册功能从 POST 请求的表单中获取 username、password、email、phone 四个字段的值，然后使用 f-string 将它们直接拼接到 INSERT 语句的 VALUES 子句中：

```
sql = f"INSERT INTO users (username, password, email, phone) \
    VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)
```

由于 f-string 不对变量中的单引号做转义处理，攻击者可以在任意一个字段中输入包含单引号 ' 的内容来闭合 VALUES 子句中的引号。例如在 username 字段中输入：

```
username = "hacker', 'pass', 'h@x.com', '123')--"
```



拼接后的 SQL 变为：

```
INSERT INTO users (username, password, email, phone)
VALUES ('hacker', 'pass', 'h@x.com', '123')--', 'irrelevant', '', '')
```

解析过程：

- 第一个字段的值 `'hacker'` 正常闭合
- `, 'pass', 'h@x.com', '123'` 提供了 password、email、phone 三个字段的值
- `)` 闭合了 VALUES 子句，INSERT 语句结构变得完整
- `--` 注释符将后续表单传入的 `', 'irrelevant', '', '')` 全部注释掉
- 最终数据库执行了一条语法完整的 INSERT 语句，插入了攻击者所控制的数据

### 4.3 攻击方式

POST /register

参数：

```
username=hacker', 'pass', 'h@x.com', '123')--
password=irrelevant (被 -- 注释掉, 不生效)
email=(被注释掉, 不生效)
phone=(被注释掉, 不生效)
```

作用：向 users 表插入一条 username="hacker", password="pass", email="h@x.com", phone="123" 的用户记录。

### 4.4 SQL 执行分析

攻击前 SQL（开发人员期望的 SQL）：

```
INSERT INTO users (username, password, email, phone)
VALUES ('hacker', 'pass', 'h@x.com', '123')--', 'irrelevant', '', '')
```

攻击后 SQL（f-string 实际拼接的结果）：

```
INSERT INTO users (username, password, email, phone)
VALUES ('hacker', 'pass', 'h@x.com', '123')--', 'irrelevant', '', '')
```

数据库执行方式：

SQLite 解析 SQL 时：

1. 读取到 VALUES ('hacker', 'pass', 'h@x.com', '123') 发现结构完整
2. `)` 与 INSERT 语句开头的 `(` 配对，VALUES 子句结束
3. `--` 开始单行注释，后续内容 `', 'irrelevant', '', '')` 被忽略
4. 执行 INSERT，插入一条完整的数据

为什么执行成功：

1. 攻击者输入中的 `'` 闭合了 username 字段的引号
2. 输入中的 `, 'pass', 'h@x.com', '123'` 提供了完整的 3 个额外字段值
3. 输入中的 `)` 闭合了 VALUES 子句的左括号

4. 输入中的 `--` 注释掉了剩余的不合法 SQL
5. 最终 SQL 语法完整，数据库正常执行 INSERT

## 4.5 漏洞影响

- 任意数据写入：攻击者可向 users 表插入任意用户名、密码、邮箱和手机号。
- 绕过注册限制：攻击者可以绕过前端的格式校验，通过注入直接控制插入的字段值。

## 5 漏洞 SQL-001：初始化 INSERT 注入

---

### 5.1 漏洞位置

|      |           |
|------|-----------|
| 文件   | app.py    |
| 函数   | init_db() |
| 调用时机 | 应用启动时     |

对应代码（修复前）：

```
def init_db():
    ...
    default_users = [
        ("admin", "admin123", "admin@example.com", "13800138000"),
        ("alice", "alice2025", "alice@example.com", "13900139001"),
    ]
    for u, p, e, ph in default_users:
        sql = f"INSERT OR IGNORE INTO users (username, password, email, phone) \
            VALUES ('{u}', '{p}', '{e}', '{ph}')"
        c.execute(sql)
    conn.commit()
    conn.close()
```

### 5.2 漏洞分析

该漏洞存在于数据库初始化函数中，u、p、e、ph 的值来自硬编码的元组，并非用户输入，因此实际风险较低。但是使用 f-string 拼接 SQL 本身就是不安全的编码习惯，如果后续维护中将默认用户数据改为从外部文件或配置文件读取，就会立即变成真正的 SQL 注入风险。本次修复一并将其改为参数化查询。

## 6 漏洞复现

### 6.1 测试环境

|         |                                |
|---------|--------------------------------|
| 操作系统    | Kali Linux                     |
| Web 服务器 | Flask 开发服务器 ( 127.0.0.1:5000 ) |
| 数据库     | SQLite 3 ( data/users.db )     |
| 测试工具    | curl、Burp Suite                |

### 6.2 复现 SQL-002 ( 搜索 UNION SELECT 注入 )

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| 请求方式    | GET                                                                    |
| 请求      | /search?                                                               |
| URL     | keyword=%27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%27138%27-- |
| Payload | ' UNION SELECT 1,'inj','inj@x.com','138'--                             |

```
# 先登录获取 Session Cookie
curl -X POST http://127.0.0.1:5000/login \
  -d "username=admin&password=admin123" -c /tmp/cookies.txt
```

```
# 执行 UNION SELECT 注入
curl -b /tmp/cookies.txt \
```

```
"http://127.0.0.1:5000/search?keyword=%27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%27138%27--"
```

# 测试结果

搜索结果中出现 username="inj", email="inj@x.com" 的自定义数据

确认 UNION SELECT 注入成功。

[Burp Suite 请求/响应截图 — UNION SELECT 注入]

[浏览器搜索结果页面截图 — 显示自定义 inj 用户数据]

## 6.3 复现 SQL-002 (搜索 OR 注入)

|         |                                           |
|---------|-------------------------------------------|
| 请求方式    | GET                                       |
| 请求 URL  | /search?keyword=%27%20OR%20%271%27%3D%271 |
| Payload | ' OR '1'='1                               |

```
curl -b /tmp/cookies.txt \  
"http://127.0.0.1:5000/search?keyword=%27%20OR%20%271%27%3D%271"
```

# 测试结果

搜索结果中显示 users 表全部用户：

admin | admin@example.com

alice | alice@example.com

testuser | test@test.com

newuser | new@test.com

确认 OR 注入成功，全部用户数据泄露。

[Burp Suite 请求/响应截图 — OR 注入]

[浏览器搜索结果页面截图 — 显示全部用户]

## 6.4 复现 SQL-003 (注册 INSERT 注入)

|                       |                                      |
|-----------------------|--------------------------------------|
| 请求方式                  | POST                                 |
| 请求 URL                | /register                            |
| Payload (username 字段) | hacker', 'pass', 'h@x.com', '123')-- |

```
curl -X POST http://127.0.0.1:5000/register \  
-d "username=hacker', 'pass', 'h@x.com', '123')--&password=irrelevant"
```

# 测试结果

返回 302 跳转到 /login?msg=注册成功，请登录

数据库中插入了 username="hacker" 的用户记录

注入语句成功改变了 SQL 的结构。

[Burp Suite 请求/响应截图 — 注册注入]

[浏览器注册页面截图]

## 7 修复方案

### 7.1 修复 SQL-002 (搜索 SELECT 注入)

修复前：

```
search_keyword = request.args.get("keyword", "")
if search_keyword:
    sql = f"SELECT id, username, email, phone FROM users \
        WHERE username LIKE '{search_keyword}%' \
        OR email LIKE '{search_keyword}%'"
    c.execute(sql)
```

修复后：

```
search_keyword = request.args.get("keyword", "")
if search_keyword:
    sql = "SELECT id, username, email, phone FROM users \
        WHERE username LIKE ? OR email LIKE ?"
    like_param = f"%{search_keyword}%"
    c.execute(sql, (like_param, like_param))
```

为什么使用参数化查询：参数化查询使用 `?` 占位符代替实际的数据值。SQLite 在收到 SQL 语句后，会先对语句进行编译生成执行计划，此时 SQL 的结构（SELECT 哪些列、WHERE 条件如何组合）被固定下来。然后 SQLite 再将参数值（`like_param`）传入已经编译好的执行计划中执行。用户输入的值仅作为数据参与 LIKE 的模式匹配，不会被解释为 SQL 代码。

为什么参数化查询可以阻止 SQL 注入：以 UNION SELECT 注入为例，修复前 `keyword = 'UNION SELECT 1,2,3,4--'` 被直接拼入 SQL，使得 UNION SELECT 成为 SQL 语句的一部分。修复后，同一个 keyword 被整体放入 `like_param`，传入 SQLite 时相当于执行：

```
SELECT ... WHERE username LIKE '%\' UNION SELECT 1,2,3,4--%'
SQLite 将 LIKE 的模式解释为：匹配用户名中包含 "' UNION SELECT 1,2,3,4--" 字符串的记录
UNION SELECT 关键字失去了 SQL 指令的功能，退化为普通文本。
```

### 7.2 修复 SQL-003 (注册 INSERT 注入)

修复前：

```
sql = f"INSERT INTO users (username, password, email, phone) \
      VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)
```

修复后：

```
sql = "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)"
c.execute(sql, (username, password, email, phone))
```

为什么使用参数化查询：修复后 `VALUES` 子句使用 4 个 `?` 占位符，`username`、`password`、`email`、`phone` 作为参数传入。SQLite 在编译时固定了“向 `users` 表的 4 个字段插入 4 个值”的结构，参数中的任何内容都只能作为字段的值，不能改变 SQL 的语法结构。

为什么参数化查询可以阻止注入：攻击者通过 `username` 输入 `hacker', 'pass', 'h@x.com', '123')--` 时，修复前该字符串会闭合引号并插入额外的 SQL 语法。修复后，该字符串被整体作为 `username` 字段的值存储到数据库中，单引号和括号被自动转义，不会破坏 `VALUES` 子句的结构。搜索验证时可以看到该字符串被完整地存储在 `username` 字段中。

### 7.3 修复 SQL-001（初始化 INSERT）

修复前：

```
sql = f"INSERT OR IGNORE INTO users (...) VALUES ('{u}', '{p}', '{e}', '{ph}')"
c.execute(sql)
```

修复后：

```
sql = "INSERT OR IGNORE INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)"
c.execute(sql, (u, p, e, ph))
```



## 8 修复前后对比

| SQL 构造方式                   | Payload                                       | 修复前结果           | 修复后结果         | 是否成功      |
|----------------------------|-----------------------------------------------|-----------------|---------------|-----------|
| f"...LIKE '%{k}%'"         | ' OR '1'='1                                   | 返回全部用户          | 无搜索结果         | ✓         |
| f"...LIKE '%{k}%'"         | ' UNION SELECT 1,'inj','inj@x.com','138'--    | 插入自定义数据 inj     | 无搜索结果         | ✓         |
| f"...LIKE '%{k}%'"         | ' ORDER BY 5--                                | ORDER BY 执行成功   | 无搜索结果         | ✓         |
| f"...VALUES ('{u}','{p}')" | username=hacker', 'pass', 'h@x.com', '123')-- | SQL 结构被篡改, 注入执行 | 注入内容被当作普通文本存储 | ✓         |
| ? 参数化                      | keyword=admin (正常搜索)                          | 返回 admin 用户     | 返回 admin 用户   | ✓<br>功能正常 |
| ? 参数化                      | 正常注册新用户                                       | 注册成功            | 注册成功          | ✓<br>功能正常 |

## 9 修复验证

### 9.1 验证方法

使用与漏洞复现阶段完全相同的 Payload 和请求方式，对修复后的系统进行测试。若修复有效，相同的注入 Payload 不应再产生非预期的数据库查询结果，同时正常的业务功能应不受影响。

### 9.2 验证结果

| 测试项目                                          | 修复前结果       | 修复后结果             | 防御是否成功 |
|-----------------------------------------------|-------------|-------------------|--------|
| ' OR '1'='1                                   | 全部用户泄露      | 无搜索结果             | ✓      |
| ' UNION SELECT 1,'inj','inj@x.com','138'--    | 插入自定义数据     | 无搜索结果             | ✓      |
| ' ORDER BY 5--                                | 列数探测成功      | 无搜索结果             | ✓      |
| username=hacker', 'pass', 'h@x.com', '123')-- | SQL 结构被篡改   | 注入内容被当作普通字符串存入数据库 | ✓      |
| 正常搜索 keyword=admin                            | 返回 admin 用户 | 返回 admin 用户       | ✓ 功能正常 |
| 正常注册新用户                                       | 注册成功        | 注册成功              | ✓ 功能正常 |
| 正常登录 admin                                    | 登录成功        | 登录成功              | ✓ 功能正常 |

### 9.3 修复验证：UNION SELECT 注入（修复后）

```
curl -b /tmp/cookies.txt \
```

```
"http://127.0.0.1:5000/search?keyword=%27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%271
```

响应：无搜索结果（表格为空），搜索关键词显示为完整注入语句字符串

验证结论：SQLite 将 "' UNION SELECT 1,'inj'..." 整体作为 LIKE 的匹配模式

去匹配用户名，没有找到匹配项，返回空结果。

UNION SELECT 注入已失效 

## 9.4 修复验证：OR 注入（修复后）

```
curl -b /tmp/cookies.txt \  
"http://127.0.0.1:5000/search?keyword=%27%20OR%20%27%3D%27"
```

响应：无搜索结果（表格为空）

验证结论：SQLite 将 "" OR '1'='1" 整体作为 LIKE 的匹配模式，  
搜索用户名中包含该字符串的用户，没有找到匹配项。

由于参数化查询将输入视为数据而非 SQL 代码，

OR 关键字失去了 SQL 指令的功能。

OR 注入已失效 

## 9.5 修复验证：注册注入（修复后）

```
curl -X POST http://127.0.0.1:5000/register \  
-d "username=hacker', 'pass', 'h@x.com', '123')--&password=irrelevant"
```

搜索验证：curl -b /tmp/cookies.txt \  
"http://127.0.0.1:5000/search?keyword=hacker"

响应表格中显示：

```
ID=5  username=hacker', 'pass', 'h@x.com', '123')--  
email=h@x.com  phone=123
```

验证结论：注入 payload "hacker', 'pass', 'h@x.com', '123')--"

被完整地作为 username 字符串存入数据库，未改变 SQL 结构。

参数化查询将输入整体视为一个字段的值，

单引号和括号被自动转义为字符串内容。

注册注入已失效 

## 10 总结

### 10.1 SQL 注入产生原因

- 使用 Python 的 **f-string** 将用户输入直接拼接到 SQL 语句中，f-string 不做任何转义处理
- 搜索功能中，keyword 从 URL 参数获取后直接拼入 LIKE 子句
- 注册功能中，表单字段直接拼入 VALUES 子句
- 唯一依赖的"防御"仅为"假设用户不会输入恶意内容"，这是完全不可靠的

### 10.2 修复方法

- 将所有 3 处 SQL 语句改为 参数化查询，使用 SQLite 的 `?` 占位符
- SELECT 查询：将 `LIKE '%{keyword}%'` 改为 `LIKE ?`，`%keyword%` 作为参数传入
- INSERT 插入：将 `VALUES ('{user}', '{pw}')` 改为 `VALUES (?, ?, ?, ?)`，字段值作为参数元组传入
- 参数化查询将 SQL 的结构与数据分离，用户输入只能作为数据，不会改变 SQL 语法

### 10.3 修复效果

| 测试项             | 结果     |
|-----------------|--------|
| OR '1'='1 注入    | ✓ 防御成功 |
| UNION SELECT 注入 | ✓ 防御成功 |
| ORDER BY 注入     | ✓ 防御成功 |
| 注册 INSERT 注入    | ✓ 防御成功 |
| 正常搜索功能          | ✓ 正常   |
| 正常注册功能          | ✓ 正常   |
| 正常登录功能          | ✓ 正常   |

修复后，全部 4 种 SQL 注入攻击方式均已失效，正常业务功能完好无损。

## 10.4 实验收获

1. 理解了 SQL 注入的产生原理：用户输入拼接 SQL 改变了语法结构
2. 掌握了代码审计的方法：通过查找 f-string、字符串拼接模式发现 SQL 注入漏洞
3. 学会了参数化查询的修复技术：使用 `?` 占位符分离 SQL 结构与数据
4. 通过修复前后对比验证，证实参数化查询能够彻底防御 SQL 注入，同时不影响业务功能

---

— 报告结束 · 2026 年 7 月 8 日 —